

BLOC B • SÉANCE 4

Faire tourner un modèle

Sur sa propre machine, sans budget



SÉANCE 4 • ATELIER

Faire tourner un modèle sur sa machine

Ollama, LM Studio, quantization : personne ne repart bloqué

Au programme aujourd'hui

1

Installer

Ollama, et un premier modèle qui tourne

2

Comprendre

La quantization, ou comment 16 Go tiennent dans 5

3

Dimensionner

Quel modèle pour quelle machine

4

Appeler

Le serveur local, en HTTP, depuis votre code

Quatre étapes, et le socle est en place

1

Installer

Ollama s'installe comme n'importe quel outil en ligne de commande.

2

Télécharger

``ollama pull`` récupère les poids une fois pour toutes.

3

Essayer

``ollama run`` ouvre un chat dans le terminal, hors ligne.

4

Servir

Un serveur HTTP écoute en local sur le port 11434.

Les commandes de l'atelier

```
# 1. installation (macOS / Linux ; installeur .exe sous Windows)
curl -fsSL https://ollama.com/install.sh | sh
```

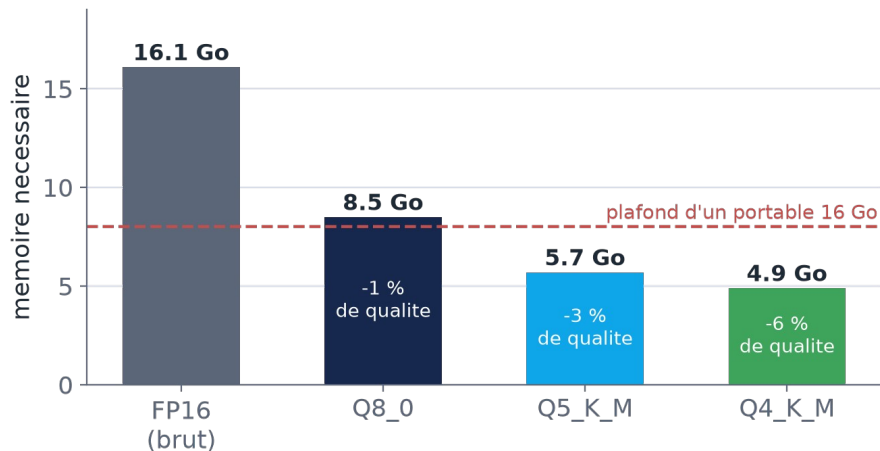
```
# 2. le socle impose a toute la classe (environ 2,2 Go)
ollama pull qwen2.5:3b
ollama pull paraphrase-multilingual
```

```
# 3. verifier que tout repond
ollama list
ollama run qwen2.5:3b "Bonjour, tu tournes en local ?"
```

Lancez les téléchargements chez vous : le réseau de l'école ne supportera pas trente `pull` simultanés.

Comment un modèle de 16 Go tient dans 5

Un modele 8B, quatre precisions : seul Q4 tient partout



Quantizer, c'est stocker chaque poids sur moins de bits : 16, puis 8, puis 4.

Q4_K_M est le compromis retenu par défaut : divisé par trois, presque aussi bon.

Quel modèle votre machine peut-elle tenir

RAM disponible	Modèle de travail	Ce que ça donne
8 Go	llama3.2:1b	ça tourne, qualité juste
8 à 16 Go	qwen2.5:3b	le socle du cours
16 Go et plus	qwen2.5:7b	confortable, plus lent
Machine trop juste	serveur de classe	identique, via le réseau

Le modèle d'embedding pèse 562 Mo : il passe sur toutes les machines, y compris celles du serveur de classe.

Deux API pour le même serveur local

C'est la seconde qui nous intéresse pour tout le reste du module.

```
# API native Ollama
curl http://localhost:11434/api/generate -d '{...}'

# API compatible OpenAI : le meme contrat que le cloud
curl http://localhost:11434/v1/chat/completions \
  -H 'Content-Type: application/json' \
  -d '{"model": "qwen2.5:3b",
      "messages": [{"role": "user", "content": "Bonjour"}]}'
```

Le code que vous écrirez fonctionnera tel quel face à un fournisseur cloud : seule l'URL de base change.

Personne ne reste bloqué

- Le poste enseignant sert un modèle plus gros sur le réseau de la salle.
- Une seule variable d'environnement change, et le code reste identique.
- C'est déjà une architecture client/serveur : nous y reviendrons.



OLLAMA_BASE_URL

En local : `http://localhost:11434`. En salle : `http://<ip-enseignant>:11434`. Rien d'autre ne bouge.

Mettre son poste en état de marche

- 1 Installer Ollama et vérifier `ollama list`.
- 2 Télécharger le socle : qwen2.5:3b et paraphrase-multilingual.
- 3 Lancer `node tp/00\_setup/check.mjs` et lire le diagnostic.
- 4 Noter le débit obtenu en tokens par seconde sur sa machine.
- 5 Si la machine ne suit pas : basculer sur le serveur de classe.

Objectif

Chacun repart avec un modèle qui tourne et un chiffre de référence pour la séance 6.

Un modèle sur sa machine

- ✓ Ollama installe, télécharge et sert un modèle en trois commandes.
- ✓ La quantization divise la mémoire nécessaire par trois pour presque aucune perte.
- ✓ On dimensionne le modèle sur la RAM disponible, pas sur l'envie.
- ✓ Le serveur local parle la même API que le cloud : le code est portable.